PYMASTER INDIA · #2

How Python Runs Your Code

Interpreter, Bytecode, REPL, and the Implementations

Go under the hood and see what happens between pressing Run and seeing your output. Understanding this makes debugging far easier.

Student Notes & Concept Guide
codeWithShivah · Rajeev Ranjan Pandey
PyMaster India Series

What You Will Learn

- What an interpreter actually is
- Compiled vs interpreted languages
- How .py files are executed — the full pipeline
- Using the Python REPL (interactive shell)
- Python vs CPython vs PyPy

> **Where we left off:** In #1 you installed Python, set up VS Code, and ran `hello.py`. You saw that it printed three lines in the order you wrote them. This video answers the question that was skipped: *what actually happened* when you pressed Run — and why knowing that will make your errors far less mysterious.

1. What Is an Interpreter?

Your computer's processor understands only machine code — raw binary instructions. It has no idea what `print("Hello")` means. Something has to stand between your English-like Python and the processor's binary. That something is the **interpreter**.

An interpreter is itself just a program. When you installed Python in #1, you installed an interpreter — a piece of software whose entire job is to read Python code and carry out what it says.

The translator analogy

Imagine you are giving a speech in Hindi to an audience that speaks only Japanese. Two approaches:

- Hire a translator who converts your whole speech into Japanese in advance, prints it, and hands it over. That is **compilation**.
- Bring a live interpreter who stands beside you and translates each sentence as you say it. That is **interpretation**.

The live interpreter starts immediately with no preparation, but works a little slower sentence by sentence. The pre-translated document takes time to prepare, but once ready it reads at full speed. Neither is better — they trade different things.

> **Why this matters for debugging.** A live interpreter stops the moment it hits a sentence it cannot translate — and everything before that point has already been delivered. That is exactly why a Python error shows output up to the failing line and nothing after it. Section 4 makes that concrete.

2. Compiled vs Interpreted Languages

	Interpreted (Python)	Compiled (C, C++, Java)
Before running	Nothing — just run the file	A separate build step produces an executable
Time to test a change	Instant — save and run	Rebuild first, then run
When errors appear	When execution reaches that line	Many caught at build time, before running
Raw speed	Slower	Faster
Portability	Same file runs anywhere Python is installed	Usually rebuilt per platform
Typical use	Scripting, data science, web backends, automation	Operating systems, games, embedded, performance-critical work

The error-timing difference, concretely

```
# Python — this file RUNS, prints, and only THEN fails
print("Line 1 works")
print("Line 2 works")
print(undefined_variable)   # fails here
print("Line 4 never runs")

# Output:
# Line 1 works
# Line 2 works
# NameError: name 'undefined_variable' is not defined
```

A compiled language would typically refuse to build this at all — you would get the error before a single line executed. Python ran two-thirds of it first. This is the trade in its purest form: Python lets you start instantly, at the cost of finding some mistakes only when execution reaches them.

> **The practical consequence.** Code that "worked fine" may still contain a mistake in a branch that never ran. A typo inside an `if` block that is rarely true can sit undetected for a long time. This is precisely why testing matters more in Python than in languages where the compiler catches such things upfront.

Python is not purely interpreted

The textbook split is a simplification. Python actually does both — it compiles, then interprets. That is section 3.

3. How .py Files Are Executed

When you run `python hello.py`, four things happen in order.

Step 1 — Source

Your `.py` file, plain text, exactly as you typed it.

Step 2 — Compile

Python reads the whole file and checks its grammar. If a bracket is unclosed or a colon is missing, it stops here and reports a **SyntaxError** — and importantly, *nothing runs at all*, not even line 1.

```
print("This will NOT print")
print("Neither will this"
#                            ^ missing bracket

# SyntaxError: '(' was never closed
# Zero output. The compile step failed, so execution never began.
```

Step 3 — Bytecode

The result of compiling is **bytecode** — a compact set of instructions that is not your source code and not machine code, but something in between. Python caches it in a `__pycache__` folder as a `.pyc` file so that re-running an unchanged file can skip the compile step.

> **You have probably already seen this.** A `__pycache__` folder appearing next to your files is normal and safe. It is generated, never edited by hand, and belongs in `.gitignore` — which is exactly why it appears there in later videos.

Step 4 — Execute

The **Python Virtual Machine** (PVM) reads the bytecode one instruction at a time and carries it out. This is the loop that is genuinely "interpreting," and it is where runtime errors — `NameError`, `TypeError`, `ZeroDivisionError` — occur.

The two error families

	Syntax errors	Runtime errors

Caught at	Step 2 (compile)	Step 4 (execute)
Output before it	None at all	Everything up to that line
Caused by	Broken grammar	Valid code, impossible action
Examples	Missing `:`, unclosed quote or bracket	`NameError`, `TypeError`, divide by zero

> **This is the debugging shortcut.** Look at whether *any* output appeared before the error. No output at all points to a syntax problem — check brackets, quotes, and colons. Partial output means your grammar was fine and the failure is in the logic at the line named. You have narrowed the search before reading a word of the message.

4. The Python REPL (Interactive Shell)

REPL stands for Read, Evaluate, Print, Loop — the shell reads your line, evaluates it, prints the result, and loops for the next one. It is a live Python session where each line runs the instant you press Enter.

Open and leave it

```
# in the terminal (not in a .py file)
python          # Windows
python3         # macOS / Linux

# you get:
Python 3.13.1 (main, Dec  3 2024, 17:59:52)
Type "help", "copyright", "credits" or "license" for more information.
>>>

# to leave:
>>> exit()          # or quit(), or Ctrl+Z then Enter (Windows) / Ctrl+D (Mac/Linux)
```

The `>>>` prompt is how you know you are inside Python rather than in your normal terminal. If you type `python hello.py` while at `>>>`, you will get a SyntaxError — that is a terminal command, not Python code. Leave first.

The useful part — it prints without print

```
>>> 2 + 2
4
>>> "hi" * 3
'hihihi'
>>> len("PyMaster")
8
>>> name = "Amit"      # assignment shows nothing
```

```
>>> name                # but the name alone shows the value
'Amit'
```

In a `.py` file, writing `2 + 2` on its own line displays nothing — you need `print()`. In the REPL the result is echoed automatically. That is the main difference, and the reason the REPL feels faster for quick checks.

> **The REPL forgets everything.** Close it and your variables are gone — there is no file to reopen. Use it for experiments and quick checks; use `.py` files for anything you want to keep or run again. Multi-line blocks also get fiddly, since indentation must be typed by hand and one mistake means retyping the block.

Two things worth knowing

Press **up arrow** to recall previous lines instead of retyping them. And `help()` works on anything — `help(print)` or `help(len)` prints the official documentation without leaving the shell.

When to use which

Use the REPL for	Use a .py file for
Testing what a function returns	Anything longer than a few lines
Quick arithmetic or string checks	Code you want to keep or share
Exploring an unfamiliar module	Programs with functions and logic
Confirming a syntax detail	Everything in this series from #3 onward

5. Python vs CPython vs PyPy

An important distinction that confuses many beginners: **Python is a language specification** — a set of rules about what the code means. An **implementation** is an actual program that follows those rules. There is more than one.

> **The analogy:** "Python" is like the rules of chess. CPython, PyPy, and the others are different chess sets — all playing the same game, built differently.

Implementation	Written in	What it is for
CPython	C	The reference implementation and the default. What you downloaded from python.org in #1. When someone says "Python," they almost always mean this.
PyPy	RPython	Uses a JIT compiler to translate hot code paths to machine code while running. Can be substantially faster for long-running, computation-heavy programs.

Jython	Java	Runs on the JVM and can use Java libraries directly. Niche, and lags behind on version support.
IronPython	C#	Targets .NET, for integration with C# codebases. Also niche.
MicroPython	C	A stripped-down Python for microcontrollers and embedded hardware.

Check what you are running

```
import platform
print(platform.python_implementation())   # CPython
print(platform.python_version())          # 3.13.1
```

CPython vs PyPy — the honest comparison

	CPython	PyPy
Startup	Fast	Slower — the JIT needs warm-up time
Long compute loops	Slower	Often much faster
Short scripts	Better	Warm-up may cost more than it saves
C extension support	Complete	Good but not universal
Ecosystem	Everything targets it	Most things work; some do not

What you should actually use: CPython. It is the default, it is what every tutorial and job assumes, and every package is tested against it. PyPy is a tool to reach for when you have measured a real performance problem in a compute-heavy program — not a starting point. Knowing the distinction exists is the point of this section; switching implementations is not something you need to do now.

Interview note: "Is Python compiled or interpreted?" is a common question. The strong answer is *both* — source is compiled to bytecode, which the PVM then interprets. Adding that CPython is one implementation among several shows you understand the language/implementation distinction, which is what the question is really probing.

Check Your Understanding

No coding homework for this video — it is a concepts video. These checks confirm the ideas landed. Try to answer before looking back at the notes.

CONFIRM ALL SIX

CHECK 1

Explain the four steps

Say out loud what happens between `python hello.py` and your output. Name all four stages and what each produces.

Answer: source → compile → bytecode → PVM executes.

CHECK 2

Predict the output

A file has three working `print()` lines, then a line using an undefined variable, then two more prints. How many lines appear before the error? Now the same file but with a missing bracket on line 1 — how many now?

Answer: three, then zero. Runtime vs syntax error.

CHECK 3

Prove it yourself

Write both files from Check 2 and run them. Confirm your predictions were right.

CHECK 4

Find your bytecode

Run any `.py` file that imports another one of your files. Locate the `__pycache__` folder and the `.pyc` inside it. Explain in one sentence why it exists.

CHECK 5

Use the REPL properly

Open it, evaluate `7 * 6`, assign a variable, recall a line with the up arrow, run `help(len)`, and exit cleanly.

CHECK 6

Name your implementation

Run the `platform` snippet from section 5. State which implementation you are on and one reason you would *not* switch to PyPy today.

OPTIONAL — GO DEEPER

TRY 1

See the actual bytecode

Python can show you the instructions from step 3 directly:

```
import dis
def add(a, b):
    return a + b
dis.dis(add)
```

You are not expected to read it fluently. The point is that bytecode is a real, inspectable thing rather than an abstraction.

TRY 2

Time the difference

Write a loop that sums the numbers 1 to 10,000,000 and time it with the `time` module. Note the result. If you ever install PyPy, run the identical file and compare — this is what "compute-heavy" means in practice.

TRY 3

Collect error messages

Deliberately trigger four errors — a missing colon, an unclosed quote, an undefined variable, and division by zero. For each, note whether output appeared first. Keep the file; you now have a personal reference for the syntax-vs-runtime distinction.

Next up — Video #3. You now know how Python runs your code. From here we start writing it properly: variables, data types, and taking input from the user. The syntax-vs-runtime distinction from section 3 will come up constantly — it is the first thing to check whenever something breaks.